



UNIVERSITY OF ASIA PACIFIC

DEPARTMENT OF CSE

Course Code: CSE 402.

Course Title: Operating System Lab.

Experiment No: 2

Experiment Title: Implementation of Shortest Job First (SJF)
CPU Scheduling Algorithm.

Date of Submission : 8/9/2026

Submitted By:

Name : Sazid Abdullah

Student Id : 23101146

Semester : 4th Year 1st Semester

Section : C-2

Submitted To
Aita Rahman Orthi
Lecturer , UAP

Problem Statement:

Given the following processes with their Arrival Time (AT) and Burst Time (BT):

Process	AT	BT
P1	3	3
P2	2	5
P3	5	4
P4	1	3
P5	6	2

Implement **SJF (Non-Preemptive)** and **FCFS** scheduling algorithms. Calculate **CT, TAT, WT, Average TAT, and Average WT**, represent the execution sequence, and compare both algorithms to determine which performs better.

Source Code:

```
processes = [  
    ["p1", 3, 3],  
    ["p2", 2, 5],  
    ["p3", 5, 4],  
    ["p4", 1, 3],  
    ["p5", 6, 2],  
]
```

```
process = [{"pid": p[0], "at": p[1], "bt": p[2]} for p in processes]
```

```
n = len(process)
```

```
completed = []
```

```
time = 0
```

```
done = 0
```

```
is_done = [False] * n
```

```
while done < n:
```

```
    ready = [p for i, p in enumerate(process) if p["at"] <= time and not is_done[i]]
```

```
    if not ready:
```

```
        time = min(p["at"] for i, p in enumerate(process) if not is_done[i])
```

```
        continue
```

```
    current = min(ready, key=lambda p: p["bt"])
```

```
    idx = process.index(current)
```

```
    start = max(time, current["at"])
```

```
    finish = start + current["bt"]
```

```
    tat = finish - current["at"]
```

```
    wt = tat - current["bt"]
```

```
    completed.append(**current, "ct": finish, "tat": tat, "wt": wt)
```

```
    is_done[idx] = True
```

```
    time = finish
```

```
    done += 1
```

```
print("SJF")
```

```
print(f'{"Pd":<4} {"AT":<4} {"BT":<4} {"CT":<4} {"TAT":<4} {"WT":<4}')
```

```
for p in completed:
```

```
    print(f'{"p[\"pid\"]":<4} {"p[\"at\"]":<4} {"p[\"bt\"]":<4} {"p[\"ct\"]":<4} {"p[\"tat\"]":<5} {"p[\"wt\"]":<4}')
```

```
sjf_avg_tat = sum(p["tat"] for p in completed) / n
```

```
sjf_avg_wt = sum(p["wt"] for p in completed) / n
```

```
print(f'Avg TAT: {sjf_avg_tat}')
```

```
print(f'Avg WT: {sjf_avg_wt}')
```

```
print()
```

```
print("FCFS")
```

```
lst = sorted(processes, key=lambda x: x[1])
```

```
print(lst)
```

```
ct = []
```

```
tat = []
```

```
wt = []
```

```
time = 0
```

```
for p in lst:
```

```
    pid, at, bt = p[0], p[1], p[2]
```

```
    if time < at:
```

```
        time = at
```

```
    time += bt
```

```
    completion = time
```

```
    turnaround = completion - at
```

```
    waiting = turnaround - bt
```

```
    ct.append(completion)
```

```
    tat.append(turnaround)
```

```
    wt.append(waiting)
```

```
print("pid", "at", "bt", "ct", "tat", "wt")
for i in range(len(lst)):
    print(lst[i][0], lst[i][1], lst[i][2], ct[i], tat[i], wt[i])

fcfs_avg_tat = sum(tat) / len(lst)
fcfs_avg_wt = sum(wt) / len(lst)

print("avg tat =", fcfs_avg_tat)
print("avg wt =", fcfs_avg_wt)

print()
print("Comparison")

if fcfs_avg_wt < sjf_avg_wt:
    print("FCFS is better (lower avg waiting time)")
elif fcfs_avg_wt > sjf_avg_wt:
    print("SJF is better (lower avg waiting time)")
else:
    print("Both are equal in avg waiting time")

if fcfs_avg_tat < sjf_avg_tat:
    print("FCFS is better (lower avg turnaround time)")
elif fcfs_avg_tat > sjf_avg_tat:
    print("SJF is better (lower avg turnaround time)")
else:
    print("Both are equal in avg turnaround time")
```

Output:

```
Run lab-3(23101146) x
SJF
Pd  AT  BT  CT  TAT  WT
p4  1   3   4   3   0
p1  3   3   7   4   1
p5  6   2   9   3   1
p3  5   4  13   8   4
p2  2   5  18  16  11
Avg TAT: 6.8
Avg WT: 3.4

FCFS
[['p4', 1, 3], ['p2', 2, 5], ['p1', 3, 3], ['p3', 5, 4], ['p5', 6, 2]]
pid at bt ct tat wt
p4 1 3 4 3 0
p2 2 5 9 7 2
p1 3 3 12 9 6
p3 5 4 16 11 7
p5 6 2 18 12 10
avg tat = 8.4
avg wt = 5.0

Comparison
SJF is better (lower avg waiting time)
SJF is better (lower avg turnaround time)
```

Discussion:

This experiment focused on implementing two non-preemptive CPU scheduling algorithms — First Come First Serve (FCFS) and Shortest Job First (SJF) — and analyzing their behavior using a common set of processes. Each process was characterized by its Arrival Time and Burst Time, from which the Completion Time, Turnaround Time, and Waiting Time were derived.

In FCFS, processes are queued and executed strictly in the order they arrive, regardless of how long each one takes to finish. This makes it easy to implement but can lead to longer average waiting times, especially when a lengthy process arrives before several shorter ones (a scenario known as the convoy effect). SJF, on the other hand, always picks the process with the smallest burst time among those currently available, which minimizes the average waiting time in most cases. However, since SJF requires knowledge of burst times in advance, it is less practical for real-time systems where burst duration isn't known beforehand. After computing CT, TAT, and WT for each process under both algorithms, the average TAT and WT were compared. The results confirmed the theoretical expectation — SJF produced lower average waiting and turnaround times than FCFS for the given process set, since prioritizing shorter jobs reduces the cumulative wait experienced by processes stuck behind longer ones.

Conclusion:

Through this experiment, both FCFS and SJF non-preemptive scheduling algorithms were implemented and their performance evaluated based on standard scheduling metrics. The results showed that SJF outperforms FCFS in terms of average waiting time and turnaround time, since it always executes the shortest available job next. FCFS, while easier to implement and fair in terms of arrival order, does not account for job length and can therefore result in inefficient scheduling under certain conditions. This comparison highlights an important trade-off in operating systems design — between simplicity (FCFS) and optimization (SJF) — and reinforces why scheduling algorithm choice significantly impacts overall system throughput and responsiveness.